Assay: Claims That Decay With the Code

Content-Addressed Evidence Graphs for Accountable AI-Assisted Software Delivery

Om Shankar Tiwari
Applied AI Technical Lead, Google
California, United States
https://orcid.org/0009-0001-2247-0383

Tangi Vass
Co-Founder & CPTO, OMNI3ai
Lyon, France
https://orcid.org/0009-0009-0414-6190

Gagan Deep Singh
Founder, Glinr Studios and theSVG.org
Texas, United States
https://orcid.org/0009-0002-0579-6514

Abstract: ***AI coding agents fail in two coupled ways. They spend most of their context window rediscovering where things live, and they assert success without evidence when the work gets hard. Repository indexes address the first with cheap context, and orchestration frameworks with adversarial review address the second with accountability. Both describe the same object, the structure of the codebase, at two timescales: what is true of the code now, and what was verified to be true, at which revision, by whom.*** **ASSAY** ***makes that observation operational. Every claim an agent makes (*****tests pass, no secrets, behavior preserved****) is bound to the Merkle hash of the dependency cone of the code it covers, so the claim is stale exactly when that code or anything it depends on changes. We show the binding is sound and minimal, and that the blast radius of a change is precisely the set of claims it invalidates. On the graph we place a risk-proportional evidence obligation, a bounded review protocol with separation of duties, and a merge gate that consults no model: coverage, freshness, signatures, exit codes, plausibility, evidence monotonicity (the mechanical form of "do not delete the failing test"), and review status.*** **ASSAY** ***is a dependency-free Python tool with an MCP server. On five public repositories a 600-token brief costs 14× to 114× less than an exploration proxy, warm rebuilds are up to 5× faster than cold ones, cone binding re-verifies 7.9% to 81.9% of claims where repository binding re-verifies all of them while per-module binding misses 23% to 68% of required invalidations, and the gate blocks 9 of 9 scripted adversarial behaviours while admitting the honest ones. Every number in this paper is generated by the released scripts.***

Index Terms: ***AI coding agents, code provenance, Merkle trees, dependency graphs, code review, software supply chain***

I. Introduction

An agent that edits code has to answer two questions before it can be trusted: where does this change go, and how do we know it worked? Both are questions about the structure of the repository. The first is about the structure now. The second is about the structure at the moment a check ran, and whether it has drifted since.

Current tooling answers them with two disconnected artifacts. Context tools produce maps. Aider's repository map [1], packers such as Repomix [2], and committed indexes such as Stacklit [3] compress a repository into a few hundred to a few thousand tokens so an agent stops reading files to learn what they contain. Governance frameworks produce process state. Liza [4] is the most disciplined example we know. It pairs every doer with an independent reviewer, makes verdicts binding, and moves the invariants that matter most (no self-approval, no forbidden transitions, no merge without review) out of the prompt and into compiled code, on the grounds that instructions bend under pressure and code does not.

Each lineage stops where the other starts. A map knows that module A depends on module B, but not that the tests covering A last ran before B changed. A task board knows a reviewer approved task 17 at commit `9b3c`, but not which modules the approval is about. A later change to an unrelated module and a change to something A depends on look identical to it: both are "after the approval". That forces a bad choice. Either every accepted verdict stays valid until someone notices otherwise, or every change invalidates everything and review starts over.

A claim about code is a claim about content, and content is content-addressable. Key a claim by the hash of the code it is about and "has this gone stale?" becomes a hash comparison rather than a judgment. The subtlety is which hash. Hashing only the named module is unsound, because a test result for A depends on B. Hashing the whole repository is sound but useless, because every edit invalidates every claim. The right key is the hash of the dependency cone of the subject, computed as a Merkle hash over the condensation of the dependency graph. Under this key staleness is sound and minimal, and the blast radius of a change, the modules whose cone hash moved, is by construction the set of claims it invalidated. Context and accountability become one computation over one graph.

This paper contributes:

- **A model.** Content-addressed modules, cone hashes over the condensation DAG, claims bound to cone hashes, and propositions establishing soundness, minimality, and the identity between blast radius and staleness frontier. A ledger of such claims is a build cache for assertions in the sense of Mokhov et al. [5].
- **A protocol.** Risk-proportional evidence obligations, a bounded review cycle with separation of duties adapted from Liza and applied per claim, and a merge gate with eight mechanical checks including evidence monotonicity.
- **A system.** ASSAY, a zero-dependency implementation with an incremental Merkle-skipped indexer for seven language families, a 600-token situation brief, a signed append-only ledger, a CLI, git hooks, and an MCP server.
- **A mechanical evaluation.** Five scripted experiments on five public repositories and on ASSAY itself, with no model in the loop, so every number reproduces to the digit.

The implementation, experiments, results, and paper source are public.[1]

The paper does not measure how much better language-model agents perform with ASSAY than without. That study needs models in the loop and is future work. The claim here is narrower and more durable: the floor that holds no matter

[1] https://github.com/OmShiv/assay-research

what the model decides can be made cheap and precise, and the structure that provides it also provides the agent's context.

II. Background

A. Context tools

The cost of an agent's first ten minutes in a repository is documented by the tools built to reduce it. Stacklit's comparison reports that packers such as Repomix produce 180k to 400k tokens for repositories of 15k to 37k lines, while its structured index produces a few hundred [3]. Aider's map ranks symbols by a reference graph and fits them into a budget [1]. These tools share a shape: parse, usually with tree-sitter [6], extract exports and imports, group into modules, rank by connectivity or recency, emit a compact artifact. Stacklit adds two ideas we keep. The artifact is committed, so every agent and human reads the same map, and regeneration is Merkle-skipped, so a git hook keeps it fresh at negligible cost. It exposes the map through the Model Context Protocol [7]. What such maps do not carry is any notion of what has been verified. When an agent claims the tests pass, the map has nothing to say about whether the claim survives the next commit.

B. Governance frameworks

Multi-agent frameworks such as MetaGPT [8], ChatDev [9], AutoGen [10], and CrewAI [11] assign roles and route messages. Specification-first methods such as Spec Kit [12] and BMAD [13] make the artifacts explicit. Failure taxonomies show why that is not enough. Cemri et al. [14] catalog specification drift, inter-agent misalignment, and weak verification, and the sycophancy and reward-tampering literature [15, 16] documents models that rewrite the test rather than the code when under pressure to appear competent.

Liza [4] answers with layers: a behavioral contract naming failure modes and countermeasures, adversarial doer and reviewer pairs whose verdicts bind, a task state machine with forbidden transitions and bounded iteration enforced by a compiled supervisor, per-task worktrees, and a blackboard [17, 18] recording every transition. Its review protocol is what we borrow most directly. A finding must be answered by accepting, countering, refuting, or escalating. Repeating an assertion is not a permitted move, and a doer who contests must name the concrete harm the requested change would cause. What Liza's blackboard does not carry is the code. Evidence is a commit SHA and an exit code, and whether a verdict still means anything after later changes is answered procedurally, by re-review after rebase, rather than structurally.

Both lineages need the dependency graph and both compute or approximate it. Only the first keeps it. The second needs it at a different moment: not when the agent reads but when the gate decides. Keying claims by cone hash lets one graph serve both moments.

III. The Evidence Graph

A. Modules, hashes, and cones

A repository state S is a finite set of files with contents, partitioned into modules $\mathcal{M}$ (in Assay, directories up to a configurable depth). With H a collision-resistant hash, the content hash of a module is the hash of its sorted (path, file hash) pairs:

$$\mathsf{h}_S(m) = H\big(\langle (f, H(\text{content}_S(f))) : f \in m\rangle_{\text{sorted}}\big).$$

Import resolution yields a dependency relation $\mathcal{D}_S \subseteq \mathcal{M} \times \mathcal{M}$, where $(a, b) \in \mathcal{D}_S$ means some file in a imports some file in b. The relation may contain cycles, as Python packages routinely do. Let $C(m)$ be the strongly connected component of m and $\text{succ}(C)$ the components reachable from C in one step of the condensation DAG.

Definition 1 (Cone and cone hash): The dependency cone of m is $\text{cone}_S(m) = \{m\} \cup \{m' : m \to^* m'\}$. The cone hash is defined on components in reverse topological order of the condensation DAG:

$$\hat{\mathsf{h}}_S(C) = H\Big(\langle (m, \mathsf{h}_S(m)) : m \in C\rangle_{\text{sorted}} \,\|\, \langle \hat{\mathsf{h}}_S(C') : C' \in \text{succ}(C)\rangle_{\text{sorted}}\Big), \quad \hat{\mathsf{h}}_S(m) = \hat{\mathsf{h}}_S(C(m)).$$

This is a Merkle tree [19] over the condensation DAG with sharing: each component's hash is computed once and referenced by every component above it. Members of a cycle share one cone hash, which is right, since they depend on each other. The root hash of the repository is the hash of all module content hashes and serves as the coarsest binding scope.

B. Claims and bindings

Definition 2 (Claim): A claim is a tuple $c = (p, S_c, \sigma, \beta, e, a, t)$: a predicate p (*tests-pass*, *lint-clean*, *no-secrets*, *behavior-preserved*, *human-approved*, and so on), a non-empty set of subject modules $S_c \subseteq \mathcal{M}$, a scope $\sigma \in \{\text{cone}, \text{self}, \text{repo}\}$, bindings $\beta : S_c \to$ hashes recorded at attestation time, evidence e, an attestor a with a role, and a timestamp t.

For scope *cone*, $\beta(s) = \hat{\mathsf{h}}_S(s)$ at the state in which the evidence was produced. Evidence is what the tool observed, not what the attestor said: the command that ran, its exit code, a hash of its full output, size and duration, and, when the output is recognisable test-runner output, the number of tests that ran. A claim whose command exited non-zero is recorded as a *failed* claim, never discarded. Failure is evidence too.

Definition 3 (Freshness): A cone-scoped claim c is fresh in state S' iff $\beta(s) = \hat{\mathsf{h}}_{S'}(s)$ for all $s \in S_c$. Otherwise it is stale.

C. Staleness is sound and minimal

Write $\Delta(S, S') = \{m : \mathsf{h}_S(m) \neq \mathsf{h}_{S'}(m)\}$ for the modules whose content changed, including modules added or removed. Import edges derive from file contents, so $\mathcal{D}_S$ and $\mathcal{D}_{S'}$ can differ only at modules in Δ. The propositions hold up to hash collisions.

Proposition 1 (Soundness): If some module in $\bigcup_{s \in S_c} \text{cone}_S(s)$ is in $\Delta(S, S')$, then c is stale in S'.

Proof sketch: Let $m \in \text{cone}_S(s) \cap \Delta$. Then $\mathsf{h}_{S'}(m) \neq \mathsf{h}_S(m)$, so the component containing m has a different own-part and hence a different cone hash. Every component from which $C(m)$ is reachable includes that hash in its successor list, directly or through a chain of components each changed by

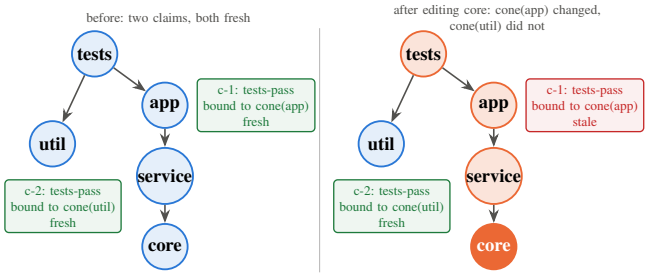


Fig. 1: Cone hashes and staleness. Arrows point from a module to what it depends on. After an edit to `core`, every module that reaches `core` (shaded) has a new cone hash, so the claim on `app` is stale and the claim on `util` is not.

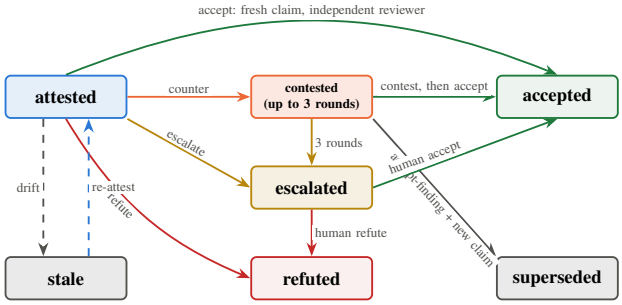


Fig. 2: Claim lifecycle. Solid arrows are ledger appends, dashed arrows are computed from the index. A non-terminal claim becomes stale on binding drift and cannot be accepted until re-attested. Refuted and superseded are terminal.

the same argument, so by induction along reverse topological order every such cone hash changes. $C(s)$ reaches $C(m)$, so $\hat{\mathsf{h}}_{S'}(s) \neq \beta(s)$. If the change instead added or removed an edge that alters $\mathrm{cone}(s)$, the file carrying the import changed, so its module lies in $\Delta \cap \mathrm{cone}_S(s)$ or in $\Delta \cap \mathrm{cone}_{S'}(s)$ and the argument applies in that direction. ∎

Proposition 2 (Minimality): If no module in $\bigcup_{s \in S_c} \mathrm{cone}_S(s)$ is in $\Delta(S, S')$, then c is fresh in S'.

Proof sketch: No module in the cone changed content, so no edge originating inside the cone changed, so $\mathrm{cone}_{S'}(s) = \mathrm{cone}_S(s)$ with identical content hashes throughout. The cone hash is a deterministic function of exactly those hashes and that structure. ∎

Together the propositions say cone binding invalidates a claim if and only if dependency semantics require re-verification. This is the correctness criterion incremental build systems use to decide whether a cached output can be reused [5]. A claim is a verifying trace whose key is (p, S_c), whose input hashes are the cone hashes, and whose output is the evidence. The gate's freshness check is the build system's "is this trace still valid" check, applied to assertions rather than artifacts.

Proposition 3 (Blast radius is the staleness frontier): Let $\mathrm{BR}(\Delta) = \Delta \cup \{m : m \to^* d \text{ for some } d \in \Delta\}$. Then $\mathrm{BR}(\Delta) = \{m : \hat{\mathsf{h}}_S(m) \neq \hat{\mathsf{h}}_{S'}(m)\}$, and the cone-scoped claims that become stale are exactly $\{c : S_c \cap \mathrm{BR}(\Delta) \neq \emptyset\}$.

Proof sketch: $m \in \mathrm{BR}(\Delta)$ iff $\mathrm{cone}_S(m) \cap \Delta \neq \emptyset$, which by Propositions 1 and 2 holds iff $\hat{\mathsf{h}}(m)$ changed. ∎

Proposition 3 is the identity that makes the two lineages one. The number an agent wants before it edits (how far does this reach?) and the number the gate wants after (which evidence is now void?) are the same set, computed by one traversal. Fig. 1 shows it on a five-module example.

D. Risk and obligation

Not every change deserves the same review. Each module carries a risk score on absolute scales so it is comparable across repositories:

$$r(m) = \tfrac{1}{2}\min\!\left(1, \frac{\text{commits}_{90d}(m)}{20}\right) + \tfrac{1}{2}\min\!\left(1, \frac{\text{fan-in}(m)}{10}\right).$$

Churn approximates instability and fan-in approximates the cost of being wrong. For a change touching Δ, the obligation level is a function of the radius fraction $|\mathrm{BR}(\Delta)|/|\mathcal{M}|$, the radius size, and the maximum risk inside the radius. Level 0 requires passing tests on the changed modules. Level 1 requires them on the whole radius. Level 2 adds a lint claim and a secrets scan on the changed modules and asks for adversarial review. Level 3 adds a human-approved claim. Fraction thresholds apply only once the radius has a minimum absolute size, so a two-module change in a five-module repository is not an escalation. Where a module's language has no test runner, the obligation asks for a build and a typecheck claim instead, so it is never unsatisfiable by construction. The thresholds are policy, stored in the index and overridable per repository. We claim only that they are explicit, monotone in radius and risk, and cheap.

IV. Protocol and Gate

A. Roles and lifecycle

Three roles appear in the ledger: doer, reviewer, and human. Any identity may attest. Only reviewers and humans may issue verdicts, and no identity may issue one on its own claim. Checked at append time, this is Liza's "no self-approval" invariant moved from task to claim granularity. A human attestation is the escalation authority itself and needs no reviewer, though it can still be refuted if its evidence is contradicted.

Fig. 2 shows the derived states. A claim starts *attested*, or *failed* if its evidence exited non-zero. A reviewer may accept it, counter it with a finding, refute it, or escalate it. A countered claim is *contested*, and the attestor must respond before any further verdict, either by accepting the finding and superseding the claim with a stronger one, or by contesting, which requires naming the concrete harm the requested change would cause. Three counters escalate automatically, and only a human verdict resolves an escalation. Refuted and superseded are terminal. Status is never stored. It is derived from the ledger each time it is needed, so the ledger cannot disagree with its own history. Staleness is orthogonal to status and derived from the index: an accepted claim whose bindings drifted is stale, and a stale claim cannot be accepted until re-attested.

B. Invariants in code

Following Liza's stance that invariants belong in code rather than prompts, each of the following is enforced mechanically,

most at ledger-append time and the rest at the gate. An attestor cannot review their own claim. A stale claim cannot be accepted. No verdict lands on a refuted or superseded claim. Only a human resolves an escalation. A contest must name a harm and a response must come from the attestor. A contested claim needs a response before the next verdict. Every ledger line carries an HMAC signature under a key held outside the repository, and an altered or unsigned line fails the gate. The gate passes only with fresh, independently accepted, plausible evidence for every requirement.

C. The gate

The gate is a function of the index, the ledger, and the changed modules. It computes the obligation, then for each required (predicate, module) pair finds the best available claim and classifies it as ok, weak, unreviewed, stale, escalated, failed, refuted, or missing. It passes iff every pair is ok and every signature verifies. Three plausibility checks stand between an accepted claim and ok, all floors rather than proofs. A *tests-pass* command must look like a test runner that can test the subject's language. It must have produced output. And the number of tests that ran must not drop below what earlier evidence on an overlapping subject reported, unless a fresh human-approved claim covers the subject. We call the last check evidence monotonicity. It is the mechanical counterpart of the contract clause "do not delete the failing test", and it costs one integer per claim.

The gate guarantees that a merge carries fresh, signed, independently reviewed evidence for the whole blast radius, and that the evidence is at least plausible. It does not guarantee the evidence is sufficient. A reviewer can accept a weak suite, two models from one provider can share a blind spot, and a doer that can read the signing key can forge the ledger. These are the boundaries Liza states for its own verdicts [4], restated here because a mechanical floor invites over-reading.

V. Implementation

A. Index

`assay index` lists files through `git ls-files` so ignore rules apply, hashes and parses each file, resolves imports, groups files into modules, computes content and cone hashes, mines 90 days of history for activity, computes risk, detects per-language test, build, and typecheck commands, and writes `assay.json`. Python is parsed with the standard library AST. TypeScript and JavaScript, Go, Rust, Java and Kotlin, Ruby, and C use conservative regular expressions that recover import specifiers and exported names. Resolution understands relative and package imports, `go.mod` module paths, Rust crate paths and workspace crates, Java packages, and TypeScript path aliases. A tree-sitter backend can replace any parser behind the same interface. Zero dependencies keep the gate runnable anywhere Python runs.

A per-file cache keyed by size and modification time stores each file's hash and parse result, so a rebuild re-parses only files whose stat changed before recomputing the graph, the cone hashes, and the activity summary. This is Stacklit's Merkle-skip idea at file granularity. The pre-commit hook installed by `assay init --hook` runs it and stages the index, so the committed index is always current.

B. Brief, ledger, and interfaces

The brief is what an agent reads before opening a source file. It carries the project's shape (hubs, hot spots, evidence commands per language), the change under way (changed modules, radius, obligation, required evidence), the state of the evidence (stale claims, open findings, the gate verdict), and a module directory ordered by fan-in and trimmed to a token budget, 600 by default. Doer and reviewer start from the same brief. Fig. 3 shows the brief for FastAPI with a hypothetical edit to its root package.

The ledger is a JSON-lines file under `.assay/`, committed as the audit trail. Every line is signed with HMAC-SHA256 under a key read from the environment or a git-ignored file. In a supervised deployment the process that executes the agent's tool calls holds the key and the agent never sees it. Appends take an exclusive lock for a read-validate-write cycle. Entries are never modified. Superseding a claim appends a new one that names the old.

The CLI exposes `init`, `index`, `brief`, `blast`, `attest`, `review`, `respond`, `gate`, `verify`, and `mcp`. The MCP server speaks JSON-RPC over stdio and exposes the same operations as tools, so Claude Code, Cursor, or any MCP client can use ASSAY without shelling out. In continuous integration, `assay gate --base origin/main` exits nonzero when a change lacks fresh reviewed evidence for its radius. The implementation is 2,313 lines of Python with no runtime dependencies and 30 tests.

VI. Evaluation

Five questions, each answered by a scripted experiment with no model in the loop. E1: how many tokens does orientation cost with a brief, an index, exploration, or a full dump? E2: how much does Merkle skipping save on rebuilds? E3: what should a claim be bound to? E4: does the gate block the ways a doer can cheat and admit the honest paths? E5: how large is a typical change, and how is obligation distributed?

A. Setup

Five public repositories spanning five languages, chosen to overlap with the comparison set published by Stacklit: Express (JavaScript), Flask (Python), Gin (Go), Axum (Rust), and FastAPI (Python), each a shallow clone with 90 days of history, plus ASSAY itself. Table I gives sizes. Tokens are counted with tiktoken/cl100k_base. Every figure and number below is produced by the released experiment scripts and injected into this document by `paper/make_numbers.py`. A dogfooding run on a private TypeScript Electron application took one real change through attestation, independent review, and a passing gate with the project's own typecheck and build commands, and surfaced two obligation defects fixed in the released version.

```
# Assay brief: fastapi @ d5aba4195b | 195 modules, 1142 files, 113579 lines | python 1138, javascript 4
Commands: python: test pytest, typecheck mypy .; javascript: no test runner | Frameworks: fastapi, flask, pytest
Hubs (touch carefully): fastapi (fan-in 171, risk 1.00); tests (fan-in 41, risk 0.95); fastapi/openapi (fan-in 6, risk ..
Hot 90d: fastapi (38 commits); tests (18 commits); scripts (8 commits); fastapi/dependencies (7 commits); fastapi/open ..
Changed: fastapi -> blast radius 177/195 modules (91%), max risk 1.00 -> level 3 (escalate)
Dependents in radius: docs_src/additional_responses, docs_src/additional_status_codes, docs_src/advanced_middleware, d ..
Required evidence: tests-pass[docs_src/additional_responses, docs_src/additional_status_codes, docs_src/advanced_middl ..
Review: independent reviewer, adversarial, human sign-off, max 3 rounds
Modules (by fan-in):
- fastapi: FastAPI framework, high performance, easy to learn, fast to code, read | exports: FastAPI, BackgroundTasks, ..
- docs_src/additional_responses: additional_responses | exports: Item, Message, read_item(), Item, read_item(), Item,  ..
- docs_src/additional_status_codes: additional_status_codes | exports: upsert_item(), upsert_item() | deps: fastapi |  ..
- .. 192 more modules: 'assay module <path>' or the MCP tool get_module
```

Fig. 3: The brief for FastAPI with a hypothetical edit to the `fastapi` package (first lines, truncated at the right margin).

B. E1: tokens to orient

Four ways for an agent to learn where things live: a full dump of every source file, an exploration proxy consisting of a directory listing plus the ten most-imported files (a conservative stand-in for the 8 to 12 files an agent reads before acting), the modules section of `assay.json`, and the brief at its default budget.

The brief costs 592 to 635 tokens on the external corpora, which is 14× to 114× less than the exploration proxy and 216× to 1225× less than a dump. The brief's cost is flat in repository size by construction. The modules section grows with the number of modules (FastAPI's 195 modules, most of them documentation examples, cost 51,842 tokens) and is meant to be queried through `get_module` rather than read whole. The ratios match those Stacklit reports for its own index [3]. What no map carries is the radius, the obligation, and the stale claims.

C. E2: incremental indexing

A warm rebuild with nothing changed is 2× to 5× faster than a cold build (Table II), and with one edited file it re-parses exactly that file. On FastAPI (1142 files) a cold build takes 508 ms and a one-file rebuild 106 ms. The residual cost is the history scan and graph recomputation, both cacheable. For a pre-commit hook this is below the noise floor of the commit itself.

D. E3: what to bind a claim to

One claim per module. For $k \in \{1, 2, 4, 8, 16, 32\}$ we draw 100 random sets of k edited modules and compute, under three binding strategies, the fraction of claims that must be re-verified (cost) and the fraction that dependency semantics say must be re-verified but the strategy leaves fresh (missed). Repo hash binds to the root hash, self hash to the module's own content hash, and cone hash is ASSAY's default.

The result is what Propositions 1 and 2 predict, now with magnitudes (Fig. 4, Table III). At four edited modules, cone binding re-verifies between 7.9% (FastAPI) and 81.9% (Gin) of claims where repo binding re-verifies all of them, and self binding misses between 23% and 68% of the claims that must be re-verified. The spread is structural. FastAPI's graph is wide and shallow, with hundreds of example modules depending on one package, so a random edit rarely lands in a cone. Axum's workspace crates form a deep chain, so most edits do. Neither the cheap-but-unsound nor the sound-but-total strategy is acceptable for a gate. Cone binding is the only one that is both, and its cost is a property of the repository, not the tool.

E. E4: the gate against an adversarial doer

A twelve-file Python project with a real pytest suite. Each scenario starts from a clean copy. A scripted doer edits and makes claims, honestly or not, a scripted reviewer acts, and the gate decides. Table IV lists the scenarios and the mechanism that fired.

all 12 scenarios end where the design says they should. The 3 honest paths pass: a leaf edit at level 1, a hub edit at level 3 with lint, secrets scan, and human sign-off, and an escalation resolved by a human, which was correctly blocked while escalated. The 9 adversarial ones fail, each on the mechanism built for it. Two rows deserve comment. *Failing test deleted* is the reward-tampering behaviour documented in the model literature [16]. The doer breaks the code, replaces the test that would catch it with a placeholder, runs a real test command that genuinely passes, and gets an accept from a reviewer who did not look. The gate still fails, because the number of tests that ran dropped from the earlier accepted evidence and no human signed off on the removal. *Fake test command* passes a real reviewer for the same reason and fails on plausibility. The point of E4 is not that these tricks are hard to think of, but that the floor holds without anyone thinking of them at review time.

F. E5: how large is a change

Across the external corpora, between 0% and 81% of modules reach only themselves (Table V). The median single-module radius is 0.5% of modules for FastAPI and 1.1% for Axum, while the 90th percentile in Axum is 91.1%, because a handful of hubs (axum-macros/src, reached by 84 modules) sit under everything. The obligation policy follows that shape. Most edits are light or standard and the hubs escalate. A uniform review policy spends the same effort on a documentation example as on the core package. The obligation spends it where the radius is.

G. Threats to validity

Import graphs from regular expressions under-approximate dependencies (dynamic imports, reflection, generated code), which makes cone hashes under-approximate and would weaken Proposition 1 in practice. Authoritative edge sources and a tree-sitter backend are the remedy, and edge recall should be reported per repository. Module granularity is a directory,

TABLE I: E1: tokens required to orient an agent, by method. Index is the complete `assay.json` including per-file entries. Modules only is the section an agent would read whole. Gain is exploration divided by brief.

Repository	Lines	Modules	Full dump	Exploration	Index	Modules only	Brief	Gain
express	21,492	35	137,093	17,782	12,447	6,230	635	28×
flask	18,345	16	134,439	46,673	11,330	3,882	595	78×
gin	24,192	7	192,995	8,328	10,862	2,379	592	14×
axum	46,686	90	335,509	45,581	42,112	19,308	621	73×
fastapi	113,579	195	733,647	68,215	142,463	51,842	599	114×
assay	4,668	5	58,582	24,430	4,487	1,626	434	56×

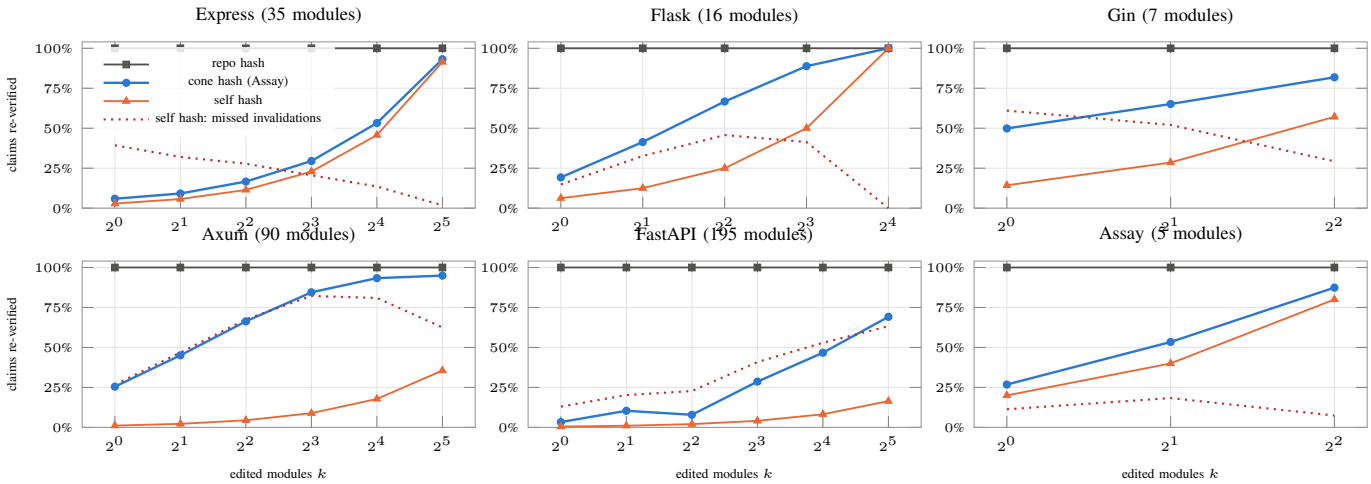


Fig. 4: E3: fraction of claims re-verified per random edit of k modules (100 trials per point) under three binding strategies. Repo-hash binding re-verifies everything. Self-hash binding is cheapest but leaves the dotted fraction of required invalidations undetected. Cone binding is the sound minimum.

TABLE II: E2: build time in milliseconds for a cold build, a warm build after one edited file, and a warm build with no change. Gain is cold divided by warm.

Repository	Files	Cold	One edit	No change	Gain (edit)	Gain (no change)
express	141	45	24	24	1.9×	1.9×
flask	83	94	32	23	3.0×	4.1×
gin	98	35	20	20	1.7×	1.7×
axum	305	62	38	34	1.6×	1.8×
fastapi	1142	508	106	93	4.8×	5.4×
assay	31	56	31	22	1.8×	2.5×

TABLE III: E3 at $k = 4$ edited modules: re-verification cost per strategy and the fraction of required invalidations the self-hash strategy misses.

Repository	Modules	Repo hash	Cone hash	Self hash	Self hash misses
express	35	100%	16.6%	11.4%	27.8%
flask	16	100%	66.7%	25.0%	45.7%
gin	7	100%	81.9%	57.1%	29.3%
axum	90	100%	66.3%	4.4%	67.6%
fastapi	195	100%	7.9%	2.1%	22.7%
assay	5	100%	87.4%	80.0%	7.4%

and a large directory inflates radii. All conditions share one tokenizer. The exploration proxy stands in for agent behaviour rather than measuring it. E4 measures a floor against scripted behaviours designed by the same people who designed the checks, so independent red-teaming is warranted. Risk and the obligation thresholds are policy.

VII. Related Work

Context for agents. Repository maps and packers [1, 2, 3], code intelligence indexes such as SCIP [20], and structure-aware retrieval in AutoCodeRover [21] and the deliberately minimal Agentless [22] all reduce the tokens an agent spends locating code. Long-context degradation [23] motivates compactness beyond cost. None bind verification to structure.

Agent frameworks and accountability. Multi-agent frameworks [8, 9, 10, 11] and agent-computer interfaces such as SWE-agent [24] and OpenHands [25] shape what agents can do. Failure taxonomies [14] and the reward-tampering literature [15, 16] explain why outcomes need verification the agent cannot game. Liza [4] is the closest system in spirit and the source of our protocol. Our contribution relative to it is the binding of verdicts to content, which turns "re-review after rebase" from procedure into arithmetic. Long-horizon measurements [26] suggest unattended runs will lengthen, raising the value of a floor that needs no attention.

Content addressing, build systems, provenance. Merkle trees [19] underlie git, Nix [27], and Bazel's action cache [28]. Mokhov et al. [5] give the vocabulary of verifying traces that our ledger instantiates for assertions. in-toto [29] and SLSA [30] attest to what ran over which inputs in a supply chain. A claim with its bindings, evidence, and signature is structurally an in-toto link moved into the inner loop, with the dependency cone as the input set. Proposition 3 is also a regression test selection oracle in the manner of Ekstazi [31], with the dependency graph shared with the agent's context.

Argumentation. The counter and contest exchange is a Toulmin argument [32]: evidence is the grounds, the cone binding is the warrant for this code state, a counter is a rebuttal, and when the binding drifts the warrant lapses.

TABLE IV: E4: scripted doer behaviours and the gate's decision. A rejected ledger entry is treated as absent, so the forged scenario reports only the signature mechanism.

Scenario	Gate	Mechanisms that fired
honest leaf edit (L1): tests on the radius, independent accept	PASS	none
honest hub edit (L3): + lint, secrets scan, human sign-off	PASS	none
no evidence at all	FAIL	coverage
evidence covers the edited module, not its dependents	FAIL	coverage
evidence produced before the edit (stale)	FAIL	freshness
doer approves own claim	FAIL	separation
forged ledger entries (claim + verdict)	FAIL	signature
code broken, real test run recorded	FAIL	exit code
code broken, fake test command	FAIL	plausibility
code broken, failing test deleted	FAIL	monotonicity
reviewer refutes the claim	FAIL	review
three rounds, escalated, human resolves	PASS	review

TABLE V: E5: single-module blast radius as a fraction of all modules, and the number of modules whose edit falls at each obligation level.

Repository	Modules	Median radius	p90 radius	L0	L1	L2	L3
express	35	5.7%	8.6%	13	22	0	0
flask	16	6.2%	87.5%	13	0	0	3
gin	7	57.1%	71.4%	1	1	1	4
axum	90	1.1%	91.1%	66	0	0	24
fastapi	195	0.5%	1.0%	149	38	0	8
assay	5	20.0%	20.0%	4	0	1	0

VIII. Limitations and Future Work

Parsers. Regular-expression extraction is the largest source of unsoundness. A tree-sitter backend and, where available, authoritative edge sources (`go list`, `tsc --listFiles`, `cargo metadata`) should replace it, and edge recall should be a reported property of every index.

Granularity. Directory modules are coarse. File-level cones are a straightforward extension. Symbol-level cones need a real call graph.

Evidence adequacy. The gate checks plausibility, not sufficiency. Coverage deltas on changed lines and per-test identities are the next mechanical checks, and per-test identities would make monotonicity exact.

Key placement. The floor depends on the signing key being outside the agent's reach. We describe the deployment pattern but do not enforce it.

Beyond tests. The same binding covers documentation (a *documents* claim goes stale when the code it describes moves), lockfiles (*no-known-vulnerabilities* bound to the lockfile hash), infrastructure plans, and monorepo federation through a dependency's version hash as a leaf. A pull request organized by claims and radius would show a reviewer what was verified, what went stale, and what is still owed.

Models in the loop. The study this paper does not contain measures resolved rate, tokens to first correct patch, tokens to gate pass, and the rate at which the gate blocks patches that pass a harness but fail hidden tests, across arms with and without the brief and the gate, on a benchmark such as SWE-bench Verified [33]. We have designed it and will preregister it.

IX. Conclusion

Context and accountability for coding agents are the same problem seen at two times. Keying claims by the Merkle hash of the dependency cone of the code they cover makes staleness a hash comparison, makes blast radius and invalidation the same set, and lets a merge gate that consults no model be both precise and cheap. ASSAY is a small, dependency-free implementation of that idea, and the experiments show the floor holding on real repositories and against scripted attacks. The best summary is also the design rule: claims should decay with the code.

Acknowledgment

The protocol and the "mechanical, not instructional" stance come from Liza. The committed, Merkle-skipped index and its MCP surface come from Stacklit. Assay shares no code with either project and would not exist without both. An AI coding assistant helped produce the figures and tables. The authors verified every claim and number and are responsible for the content.

References

[1] P. Gauthier, "Aider: Building a better repository map with tree-sitter," https://aider.chat/2023/10/22/repomap.html, 2023.

[2] K. Yamada, "Repomix: Pack your codebase into AI-friendly formats," https://github.com/yamadashy/repomix, 2024.

[3] Glincker, "Stacklit: AI-ready codebase indexing," https://github.com/glincker/stacklit, 2026, accessed September 2026.

[4] Liza contributors, "Liza: Orchestration of Accountable Agents," https://github.com/liza-mas/liza, 2026, accessed September 2026.

[5] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, p. 79:1 to 79:29, 2018.

[6] M. Brunsfeld, "Tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io, 2018.

[7] Anthropic, "Model Context Protocol," https://modelcontextprotocol.io, 2024.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber,

"MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[9] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, p. 15174 to 15186.

[10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[11] CrewAI, "CrewAI: Framework for orchestrating role-playing, autonomous AI agents," https://github.com/crewAIInc/crewAI, 2023.

[12] GitHub, "Spec Kit: Spec-Driven development toolkit," https://github.com/github/spec-kit, 2025.

[13] BMAD contributors, "BMAD method: Breakthrough Method for Agile AI-Driven development," https://github.com/bmad-code-org/BMAD-METHOD, 2025.

[14] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do Multi-Agent LLM systems fail?" *arXiv preprint arXiv:2503.13657*, 2025.

[15] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, and E. Perez, "Towards understanding Sycophancy in Language Models," in *International Conference on Learning Representations (ICLR)*, 2024.

[16] C. Denison, M. MacDiarmid, F. Barez, D. Duvenaud, S. Kravec, S. Marks, N. Schiefer, R. Soklaski, A. Tamkin, J. Kaplan, B. Shlegeris, S. R. Bowman, E. Perez, and E. Hubinger, "Sycophancy to Subterfuge: Investigating Reward-Tampering in large Language Models," *arXiv preprint arXiv:2406.10162*, 2024.

[17] H. P. Nii, "The Blackboard model of Problem Solving and the Evolution of Blackboard Architectures," *AI Magazine*, vol. 7, no. 2, p. 38 to 53, 1986.

[18] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II Speech-Understanding system: Integrating Knowledge to Resolve Uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, p. 213 to 253, 1980.

[19] R. C. Merkle, "A Digital Signature based on a Conventional Encryption Function," in *Advances in Cryptology (CRYPTO)*, 1987, p. 369 to 378.

[20] Sourcegraph, "SCIP Code Intelligence Protocol," https://github.com/scip-code/scip, 2022.

[21] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous Program Improvement," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2024, p. 1592 to 1604.

[22] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software Engineering Agents," *arXiv preprint arXiv:2407.01489*, 2024.

[23] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, p. 157 to 173, 2024.

[24] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[25] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," *arXiv preprint arXiv:2407.16741*, 2024.

[26] T. Kwa, B. West, J. Becker, A. Deng, K. Garcia, M. Hasin, S. Jawhar, M. Kinniment, N. Rush, S. Von Arx, R. Bloom, T. Broadley, H. Du, B. Goodrich, N. Jurkovic, L. H. Miles, S. Nix, T. Lin, C. Painter, N. Parikh, D. Rein, L. J. K. Sato, H. Wijk, D. M. Ziegler, E. Barnes, and L. Chan, "Measuring AI ability to complete long software tasks," *arXiv preprint arXiv:2503.14499*, 2025.

[27] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A Safe and Policy-Free System for Software Deployment," in *Proceedings of the 18th Large Installation System Administration Conference (LISA)*, 2004, p. 79 to 92.

[28] Google, "Bazel: a fast, scalable, multi-language build system," https://bazel.build, 2015.

[29] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium*, 2019, p. 1393 to 1410.

[30] OpenSSF, "SLSA: Supply-chain Levels for Software Artifacts," https://slsa.dev, 2021.

[31] M. Gligoric, L. Eloussi, and D. Marinov, "Practical Regression Test Selection with Dynamic File Dependencies," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2015, p. 211 to 222.

[32] S. E. Toulmin, *The Uses of Argument*. Cambridge University Press, 1958.

[33] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models resolve Real-World GitHub Issues?" in *International Conference on Learning Representations (ICLR)*, 2024.